Module 12
Assignment 12A: Integrating OpenAI
Laravel + OpenAI API (gpt-4o-mini)

Course: CS 85 - PHP Programming
Instructor: Vicky Seno

Repository: github.com/SergeHall/cs85-php-programming/tree/main/assignments/module12a
OpenAI quickstart: platform.openai.com/docs/quickstart

> **Outcome**
> Assignment 12A is connected to the live OpenAI API. Laravel validates the form, builds an adaptive prompt, calls gpt-4o-mini, and returns an editable draft while the API key remains on the server.

Implementation objective. Build a focused content generator that supports a blog post, meta description, or email subject line with professional, casual, and humorous tones.

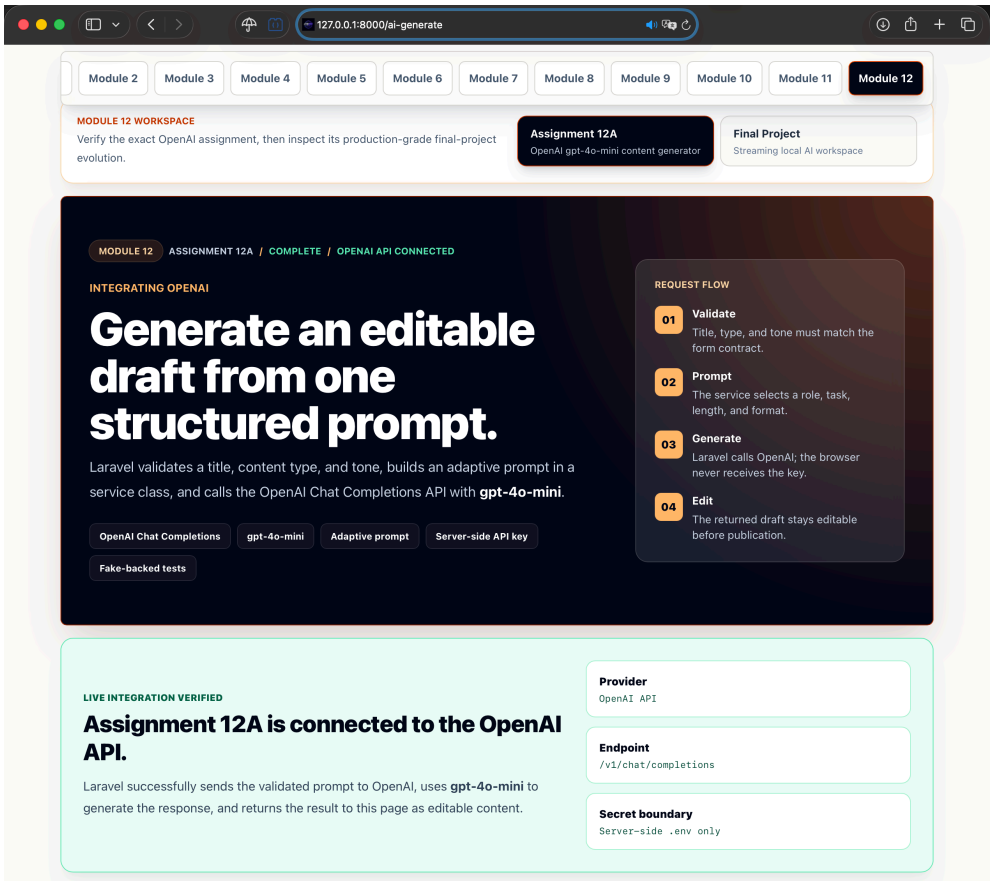
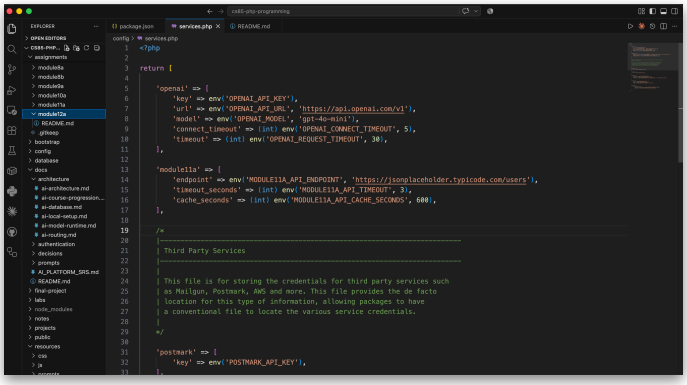


Figure 1. Assignment 12A page and verified OpenAI API boundary

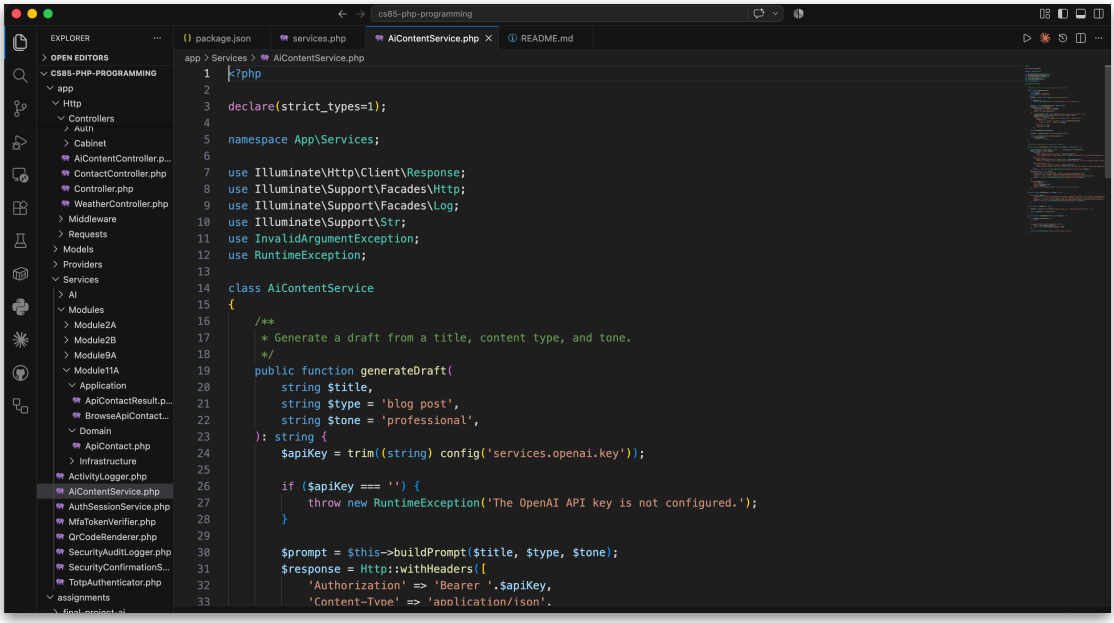
1. How the OpenAI API Was Connected

The integration follows a server-side boundary. The browser never receives the secret. Laravel reads configuration from the environment, prepares the prompt, and sends the authenticated request to OpenAI.

CODE 1 | .env and config/services.php (secret redacted)



CODE 2 | AiContentService sends the Chat Completions request



Request flow

Step	Owner	What happens
01	**Blade form**	Posts title, content type, tone, and CSRF token to Laravel.
02	**Controller**	Validates the allowlisted values and delegates to AiContentService.
03	**Service**	Builds the adaptive prompt and calls OpenAI with the server-side key.
04	**Blade result**	Receives assistant content and keeps it editable before publishing.

Security boundary. The real key is excluded from Git through .gitignore, is not printed in logs or screenshots, and is used only by the Laravel HTTP client.

2. Laravel Coordination and Browser Result

CODE 3 | Routes and controller validation

```php
<?php

declare(strict_types=1);

namespace App\Http\Controllers;

use App\Services\AiContentService;
use Illuminate\Contracts\View\View;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Throwable;

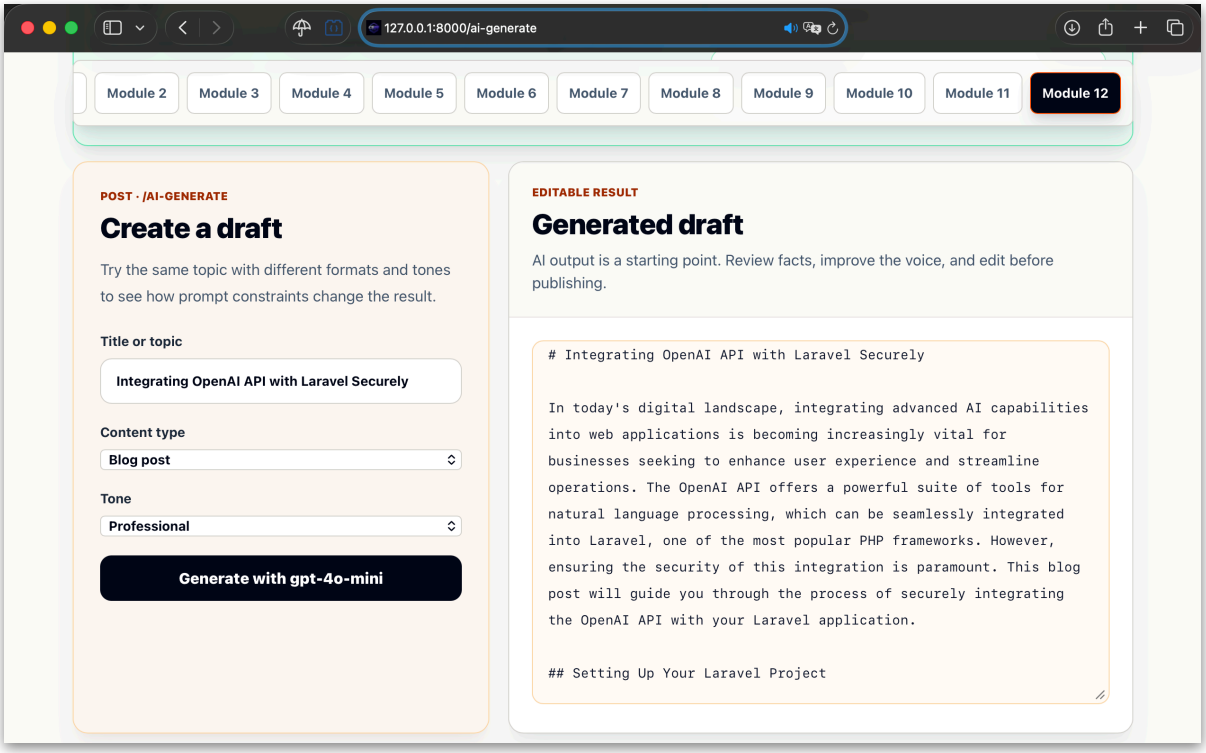
final class AiContentController extends Controller
{
    public function showForm(): View
    {
        return view('ai_form', $this->viewData());
    }

    public function generate(Request $request, AiContentService $ai): View|RedirectResponse
    {
        $validated = $request->validate([
            'title' => ['required', 'string', 'min:5', 'max:255'],
            'type' => ['required', 'in:blog post,meta description,email subject line'],
            'tone' => ['required', 'in:professional,casual,humorous'],
        ]);

        try {
            $output = $ai->generateDraft(
                $validated['title'],
                $validated['type'],
                $validated['tone'],
```

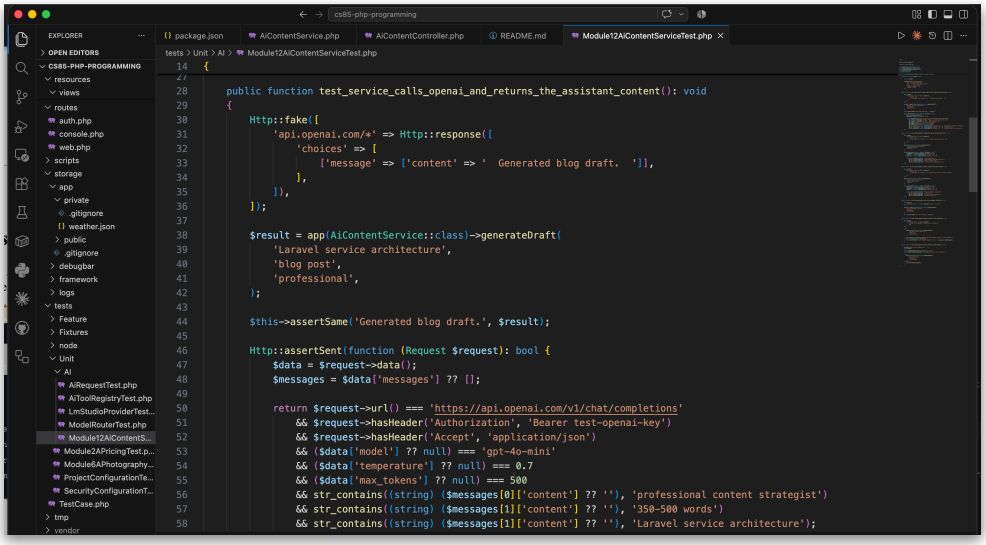
Adaptive prompting. AiContentService selects a role from the requested tone and a format constraint from the requested content type. This gives one model three focused writing modes without duplicating controller logic.

Figure 2. Live gpt-4o-mini response displayed as editable content



3. Testing, Run Instructions, and Evaluation

Automated tests do not spend API credits. Laravel HTTP fakes verify the exact endpoint, Authorization header, model, prompt roles, response extraction, and failure behavior.



```php
{
    public function test_service_calls_openai_and_returns_the_assistant_content(): void
    {
        Http::fake([
            'api.openai.com/*' => Http::response([
                'choices' => [
                    ['message' => ['content' => '  Generated blog draft.  ']],
                ],
            ]),
        ]);

        $result = app(AiContentService::class)->generateDraft(
            'Laravel service architecture',
            'blog post',
            'professional',
        );

        $this->assertSame('Generated blog draft.', $result);

        Http::assertSent(function (Request $request): bool {
            $data = $request->data();
            $messages = $data['messages'] ?? [];

            return $request->url() === 'https://api.openai.com/v1/chat/completions'
                && $request->hasHeader('Authorization', 'Bearer test-openai-key')
                && $request->hasHeader('Accept', 'application/json')
                && ($data['model'] ?? null) === 'gpt-4o-mini'
                && ($data['temperature'] ?? null) === 0.7
                && ($data['max_tokens'] ?? null) === 500
                && str_contains((string) ($messages[0]['content'] ?? ''), 'professional content strategist')
                && str_contains((string) ($messages[1]['content'] ?? ''), '350-500 words')
                && str_contains((string) ($messages[1]['content'] ?? ''), 'Laravel service architecture');
```

CODE 4 | Fake-backed OpenAI contract test

Verification summary

PASS	**Full Laravel suite** 204 tests passed with 1,677 assertions.
PASS	**Module 12 tests** Service, prompt variants, validation, and safe failures pass.
PASS	**Code style** Laravel Pint completed successfully.
PASS	**Frontend build** Vite production build completed successfully.
PASS	**Live API smoke test** OpenAI returned editable content through POST /ai-generate.

Run and test

TERMINAL



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> php artisan test --filter=Module12

   PASS  Tests\Unit\AI\Module12AiContentServiceTest
   ✓ service calls openai and returns the assistant content                0.05s
   ✓ prompt adapts to meta description and humorous tone
   ✓ prompt adapts to email subject and casual tone
   ✓ missing content uses the assignment fallback
   ✓ failed openai response throws a safe exception                        0.01s
   ✓ missing api key fails before an http request

   PASS  Tests\Feature\Module12AiContentAssignmentTest
   ✓ assignment 12a form exposes every required input                      0.02s
   ✓ generate returns the mocked services output                           0.01s
   ✓ assignment 12a validates title type and tone before calling ai        0.01s
   ✓ assignment 12a preserves input and returns a safe provider error      0.01s
   ✓ module 12 page maps assignment 12a to the final project               0.01s
   ✓ module 12 course configuration registers both deliverables            0.01s

   Tests:    12 passed (56 assertions)
   Duration: 0.17s

~/VSCode/cs85-php-programming main >

dev Task
format...
zsh
```

Conclusion

The assignment meets the required OpenAI integration goal and also adds a clean service boundary, adaptive prompts, timeout handling, safe provider errors, and fake-backed tests. The implementation is intentionally separate from the larger Final Project AI workspace, making the Module 12A rubric easy to review.